National Institute of Technology Andhra Pradesh

Department of Electronics and Communication Engineering

Course: **Machine Intelligence**
Assignment: **Assignment 2**

PROJECT REPORT

Project Title

P23 – IMDB Sentiment Analysis: BiLSTM vs TF-IDF

Team Members

No.	Name	Roll No.
1	Raj Vishwakarma	623187
2	Satyam Kumar	623172

Professor: **Vindhveshwari Prakash Singh**

Academic Year: 2026–2027
Date of Submission: 8 November 2026

Abstract

Sentiment analysis is a core Natural Language Processing task that identifies the polarity of opinions expressed in text. This report compares two approaches to binary sentiment classification on the IMDB Large Movie Review Dataset, which comprises 50,000 labelled reviews; after removal of duplicate reviews, 24,904 training reviews and 24,801 test reviews were used. The first approach represents each review with Term Frequency–Inverse Document Frequency (TF-IDF) features (20,000 features, unigrams and bigrams) and classifies it with Logistic Regression. The second approach uses a Keras tokenizer, 128-dimensional trainable word embeddings and a Bidirectional Long Short-Term Memory (BiLSTM) network with 64 units per direction (2,658,945 trainable parameters), trained with the Adam optimiser, binary cross-entropy loss, dropout of 0.5 and early stopping.

The report presents the mathematical foundations of both methods, the preprocessing and exploratory analysis, the model architectures, and the experimental results. On the test set, TF-IDF with Logistic Regression achieved an accuracy of 0.8962 and an F1-score of 0.8971, with a training time of 25.34 s. The BiLSTM reached a best validation accuracy of 0.8759 during training (64.45 s on a GPU) before over-fitting set in. However, its recorded test accuracy of 0.5714 was obtained with a tokenizer that differed from the one used during training, and it is therefore reported as provisional rather than as a valid measure of the architecture. Consequently, no claim of superiority is made for the BiLSTM. The study identifies TF-IDF with Logistic Regression as a strong, inexpensive baseline on this dataset and highlights the importance of keeping preprocessing artefacts consistent between training and evaluation in deep-learning pipelines.

Keywords: *Sentiment analysis, Natural Language Processing, TF-IDF, Logistic Regression, LSTM, BiLSTM, deep learning, IMDB dataset.*

Table of Contents

1. Introduction 1
1.1 Background 1
1.2 Motivation and Scope 1
1.3 Organisation of the Report 1
2. Problem Statement 1
3. Objectives 2
4. Dataset Description 2
5. Exploratory Data Analysis 3
5.1 Data Integrity 3
5.2 Class Distribution 3
5.3 Review Length 4
5.4 Qualitative Inspection 5
6. Text Preprocessing 5
7. Methodology 7
7.1 TF-IDF Vectorisation 7
7.2 Logistic Regression 8
7.3 Tokenisation and Padding 9
7.4 Word Embeddings 9
7.5 LSTM and Bidirectional LSTM 9
7.6 Training Objective and Optimisation 10
7.7 Evaluation Metrics 11
8. Model Architecture 11
8.1 TF-IDF with Logistic Regression 11
8.2 Bidirectional LSTM 12
9. Implementation Details 12
9.1 Software Environment 12
9.2 Implementation Stages and Their Purpose 13
10. Experimental Setup 14
11. Results and Performance Evaluation 15
11.1 TF-IDF with Logistic Regression 15
11.2 BiLSTM Training Behaviour 15
11.3 BiLSTM Test Evaluation and Validity of the Recorded Result 16
11.4 Confusion Matrices 17
11.5 Overall Performance Comparison 18
11.6 Computational Efficiency and Training Time 19
12. Comparative Analysis: TF-IDF versus BiLSTM 19
13. Custom Review Predictions 20
14. Discussion 21
15. Limitations 21
16. Conclusion 22
References 23

List of Tables

Table 1: Dataset composition before and after duplicate removal....3
Table 2: Descriptive statistics of review length (words) in the training split....4
Table 3: Text-preprocessing operations and notebook examples....5
Table 4: TF-IDF vectoriser configuration used in the notebook....8
Table 5: Ten highest TF-IDF weights for the first training review....8
Table 6: Layer-by-layer summary of the BiLSTM model (from model.summary())....12
Table 7: Software environment (versions identified from notebook outputs)....12
Table 8: Implementation stages and their purpose....13
Table 9: Experimental configuration of the two models....14
Table 10: Classification report of TF-IDF + Logistic Regression on the test set (as printed in the notebook). 15
Table 11: BiLSTM training log (values from the notebook output)....15
Table 12: Classification report of the BiLSTM on the test set, as recorded in the notebook....16
Table 13: Additional BiLSTM validation information from the training log....17
Table 14: Confusion-matrix counts and derived error rates on the test set (positive class = 1)....18
Table 15: Performance of the two models on the test set (recorded values)....18
Table 16: Computational characteristics of the two models....19
Table 17: Comparison of TF-IDF + Logistic Regression and BiLSTM....19
Table 18: Predictions of the two models on custom reviews....20

List of Figures

Figure 1: Sentiment distribution of the training split after duplicate removal....4
Figure 2: Distribution of review lengths by class (0 = negative, 1 = positive)....5
Figure 3: Overall experimental workflow of the two classification pipelines....11
Figure 4: Architecture of the BiLSTM sentiment classifier....12
Figure 5: BiLSTM training and validation accuracy (a) and loss (b), redrawn from the logged values....15
Figure 6: Confusion matrices on the test set: TF-IDF + Logistic Regression (left) and BiLSTM, recorded evaluation (right; see Section 11.3)....17
Figure 7: Comparison of test-set metrics (a) and training time (b); BiLSTM values as recorded in the notebook....18

1. Introduction

1.1 Background

Online platforms generate an enormous volume of opinionated text, including product reviews, social-media posts and film criticism. Sentiment analysis, also referred to as opinion mining, is the area of Natural Language Processing (NLP) concerned with automatically determining the polarity of such text [2], [4]. In its simplest form the task is a binary classification problem in which a document is assigned to a positive or a negative class. Reliable sentiment classifiers are used in recommendation engines, reputation monitoring, market research and the automated analysis of public opinion.

Early machine-learning studies of sentiment showed that bag-of-words features combined with linear classifiers already give strong results on movie reviews [3]. More recently, neural sequence models such as the Long Short-Term Memory (LSTM) network [9] have been applied to the same task because they can, in principle, model word order and long-range context that bag-of-words representations discard. Whether this additional modelling capacity translates into better predictive performance, and at what computational cost, is an empirical question that depends on the data, the architecture and the training procedure.

1.2 Motivation and Scope

This project compares two representative approaches on the same benchmark, the IMDB Large Movie Review Dataset [1]: (i) a traditional pipeline that represents each review as a sparse Term Frequency–Inverse Document Frequency (TF-IDF) vector and classifies it with Logistic Regression, and (ii) a deep-learning pipeline in which a Bidirectional LSTM (BiLSTM) reads a sequence of learned word embeddings. The comparison covers predictive quality (accuracy, precision, recall, F1-score and confusion matrices) as well as computational cost (training time and model size).

All results reported in this document are taken from the executed Jupyter notebook submitted with the assignment. No figure has been estimated or adjusted. Where a quantity was not recorded in the notebook, a clearly marked placeholder is used instead, and where the notebook outputs reveal a methodological inconsistency this is stated explicitly (see Section 11.3).

1.3 Organisation of the Report

Sections 2 and 3 state the problem and the objectives. Sections 4 to 6 describe the dataset, the exploratory analysis and the text preprocessing. Section 7 presents the mathematical foundations of the methods, Section 8 the model architectures, and Sections 9 and 10 the implementation and the experimental setup. Sections 11 to 14 report and discuss the results, including a comparison of the two approaches and predictions on custom reviews. Limitations and the conclusion follow in Sections 15 and 16.

2. Problem Statement

Let $D = \{(x_i, y_i)\}$, $i = 1, \ldots, n$, be a collection of movie reviews, where x_i is the raw text of the i-th review and $y_i \in \{0, 1\}$ is its sentiment label (0 = negative, 1 = positive). The goal is to learn a classifier f that maps an unseen review x to a predicted label $\hat{y} = f(x)$ so that the error on a held-out test collection is as small as possible.

Two hypothesis classes are considered. The first maps a review to a fixed-length sparse TF-IDF vector and applies a linear decision boundary (Logistic Regression). The second maps a review to a sequence

of integer word indices, embeds it, and processes it with a bidirectional recurrent network. The project addresses the following questions:

1. How accurately can each approach classify the sentiment of IMDB reviews on an independent test set?
2. How do the two approaches differ in training time, model size and ease of training?
3. What do the observed behaviours of the two models reveal about the role of word order and learned representations in this task?

3. Objectives

The objectives of the assignment are as follows:

1. To understand and explore the IMDB Movie Reviews Dataset, including class balance, data quality and review-length statistics.
2. To design and apply a text-preprocessing procedure suitable for sentiment classification.
3. To implement TF-IDF vectorisation and train a Logistic Regression classifier.
4. To implement tokenisation, padding and trainable word embeddings, and to build and train a Bidirectional LSTM classifier.
5. To evaluate both models on the same test set using accuracy, precision, recall, F1-score and confusion matrices.
6. To compare the models in terms of predictive performance, training time and model complexity.
7. To examine the behaviour of both models on custom reviews, and to discuss the advantages and limitations of each approach.

4. Dataset Description

The experiments use the IMDB Large Movie Review Dataset introduced by Maas et al. [1], accessed through the Hugging Face Datasets library [21] under the identifier stanfordnlp/imdb [22]. The corpus contains 50,000 labelled reviews divided evenly into a training split of 25,000 reviews and a test split of 25,000 reviews. Each split is balanced between the two classes. In the original construction, a review with a rating of 4 out of 10 or lower is labelled negative and a review with a rating of 7 out of 10 or higher is labelled positive; reviews with intermediate ratings are excluded, so the task consists of separating clearly polarised opinions [1]. The repository also provides an additional unlabelled split of 50,000 reviews, which was downloaded by the loading command but not used in this project.

Each record has two fields: text, the raw review including HTML mark-up such as line-break tags, and label, an integer where 0 denotes negative and 1 denotes positive. Table 1 summarises the dataset before and after the duplicate-removal step described in Section 5.1.

Table 1: Dataset composition before and after duplicate removal.

Split	Original reviews	After removing duplicates	Negative (label 0)	Positive (label 1)
Training	25,000	24,904	12,432	12,472
Testing	25,000	24,801	12,361	12,440
Total	50,000	49,705	24,793	24,912
BiLSTM training partition (80 %)	—	19,923	—	—
BiLSTM validation partition (20 %)	—	4,981	—	—

Note. Class counts for the training split were produced by the notebook (value_counts); the test-split counts are the class supports reported in the classification report. Partition sizes of 19,923 and 4,981 follow from a stratified 80/20 split of the 24,904 training reviews and agree with the 312 optimisation steps per epoch recorded in the training log (⌈19,923 / 64⌉ = 312).

5. Exploratory Data Analysis

Exploratory Data Analysis (EDA) was carried out before any modelling to verify data quality and to obtain statistics that guide design decisions such as the maximum sequence length.

5.1 Data Integrity

The training split has two columns and 25,000 rows, and the test split has the same structure. No missing values were found in the training data (0 null entries in both text and label). A duplicate check on the text column identified 96 duplicated reviews in the training split. Duplicates were removed from both splits, retaining the first occurrence, which reduced the training split to 24,904 reviews and the test split to 24,801 reviews (199 test reviews removed). Removing duplicates prevents identical reviews from appearing with excessive weight during training and avoids inflating test performance through repeated samples.

Because the duplicate removal also modifies the test split, the test results in this report are computed on 24,801 reviews rather than on the full 25,000-review benchmark and are therefore not strictly comparable to figures published for the standard test set.

5.2 Class Distribution

After duplicate removal the training split contains 12,472 positive and 12,432 negative reviews (50.1 % and 49.9 %), so the classes remain almost perfectly balanced (Figure 1). The test split is similarly balanced, with 12,440 positive and 12,361 negative reviews. Because of this balance, accuracy is a meaningful headline metric and no re-sampling or class weighting is required.

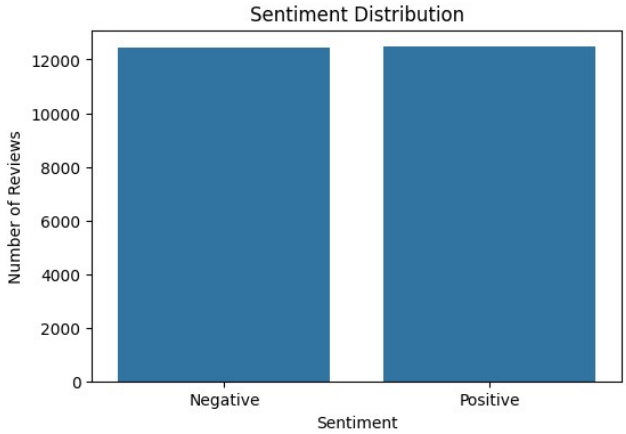


Figure 1: Sentiment distribution of the training split after duplicate removal.

5.3 Review Length

The length of each review was measured as the number of whitespace-separated words in the raw text. Table 2 and Figure 2 summarise the distribution. The distribution is strongly right-skewed: the median review contains 174 words, the mean is 234 words, and the longest review contains 2,470 words. The mean length is similar for the two classes (231.1 words for negative and 236.9 words for positive reviews), so review length alone carries little information about sentiment.

Table 2: Descriptive statistics of review length (words) in the training split.

Statistic	Value (words)
Count	24,904
Mean	234.01
Standard deviation	173.91
Minimum	10
25th percentile	127
Median (50th percentile)	174
75th percentile	285
Maximum	2,470
Mean – negative reviews	231.08
Mean – positive reviews	236.92

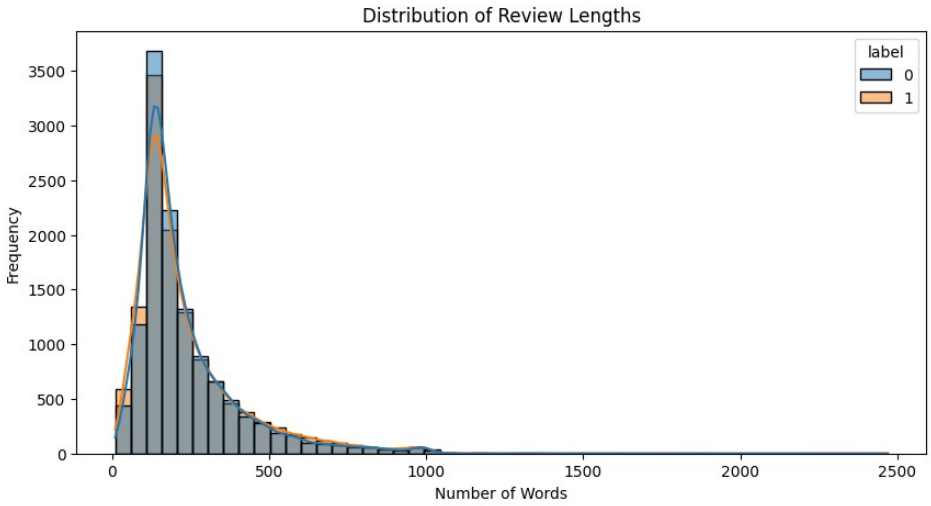


Figure 2: Distribution of review lengths by class (0 = negative, 1 = positive).

The 75th percentile (285 words) lies just below the chosen maximum sequence length of 300 tokens for the BiLSTM. Consequently most reviews fit completely within the input window, while the long tail of the distribution (reviews of several hundred words) is truncated. This observation motivated the sequence length used in Section 7.3 and is revisited as a limitation in Section 15.

5.4 Qualitative Inspection

Sample reviews of each class were printed and inspected. The raw text contains HTML line-break tags (
), mixed capitalisation, punctuation, quotation marks and numerals. For example, the first training review, a negative review of the film “I Am Curious: Yellow”, contains repeated

 tags between paragraphs. This confirmed that a cleaning stage is required before feature extraction.

6. Text Preprocessing

A single preprocessing function, preprocess_text, was written as a composition of three elementary operations and was applied identically to the training and testing reviews of both models. Table 3 lists the operations together with examples taken from the notebook.

Table 3: Text-preprocessing operations and notebook examples.

Operation	Implementation	Example (input → output)
HTML removal	BeautifulSoup (html.parser), get_text()	“Great! Amazing acting.” → “Great! Amazing acting.”
Lower-casing	str.lower()	“THIS MOVIE IS AMAZING” → “this movie is amazing”
Special-character removal	Regular expression replacing every character other than letters, digits, whitespace and apostrophes by a space	“Amazing movie!!! Rating: 10/10.” → “Amazing movie Rating 10 10”
Whitespace normalisation	Collapse runs of whitespace; strip ends	Multiple spaces and line breaks → single spaces
Combined function	preprocess_text()	“THIS movie was absolutely AMAZING!!! I don't regret watching it.” → “this movie was absolutely amazing i don't regret watching it”

Design decisions. (i) Stop-word removal was deliberately not applied. Negations and intensifiers such as “not”, “no” and “very” frequently determine polarity, and an aggressive stop-word list would remove them. (ii) Neither stemming nor lemmatisation was applied, which keeps the pipeline simple and lets the models see inflected forms such as “loved” and “loving” as distinct evidence. (iii) Apostrophes were preserved so that contractions such as “don't” remain single units.

Interaction with the tokenisers. The default token pattern of scikit-learn's TfidfVectorizer splits a contraction at the apostrophe and discards tokens of a single character, so “don't” is reduced to the token “don”; the word “not” itself is retained, and the inclusion of bigrams allows phrases such as “not good” to be represented. The Keras Tokenizer, in contrast, does not filter the apostrophe by default and therefore keeps “don't” as one token. These behaviours are properties of the libraries' defaults and were not modified in the notebook.

The cleaning step was verified on sample strings and on the first training review, and the number of missing cleaned reviews after processing was confirmed to be zero for the training split.

7. Methodology

This section presents the mathematical foundations of the two approaches. Both operate on the cleaned text produced by the procedure of Section 6.

7.1 TF-IDF Vectorisation

TF-IDF is a term-weighting scheme from information retrieval [5], [6], [7]. It represents a document as a vector with one coordinate per vocabulary term. The weight of a term grows with its frequency in the document (term frequency) and shrinks with the number of documents in which it appears (inverse document frequency), so that words that are common to all reviews receive small weights while words that are distinctive for a subset of reviews receive large weights. For a term t in a document d, the normalised term frequency is

$$tf(t,d) = \frac{f_{t,d}}{\sum_{t' \in d} f_{t',d}} \tag{1}$$

where $f_{t,d}$ is the number of occurrences of t in d. With N documents in the training collection and df(t) the number of documents that contain t, the classical inverse document frequency and the TF-IDF weight are

$$idf(t) = \log\left(\frac{N}{df(t)}\right) \tag{2}$$

$$tfidf(t,d) = tf(t,d) \times idf(t) \tag{3}$$

Variant implemented by scikit-learn. The TfidfVectorizer used in the notebook relies on the library's default settings (smooth_idf = True, sublinear_tf = False, norm = “l2”). It uses the raw count $f_{t,d}$ as the term-frequency component and a smoothed inverse document frequency,

$$idf(t) = \ln\left(\frac{1+N}{1+df(t)}\right) + 1 \tag{4}$$

so that the unnormalised weight is $x_{d,t} = f_{t,d} \cdot \text{idf}(t)$. Each document vector is then scaled to unit Euclidean length,

$$v_d = \frac{x_d}{\|x_d\|_2} \tag{5}$$

which removes the dependence of the representation on document length. The vectoriser was configured as follows (see Table 4): the vocabulary is limited to the 20,000 most frequent terms of the training corpus (max_features = 20,000); both unigrams and bigrams are extracted (ngram_range = (1, 2)); terms occurring in fewer than two documents are ignored (min_df = 2) as are terms occurring in more than 95 % of documents (max_df = 0.95). Bigrams allow short local patterns such as “not good” or “well worth” to become features even though the model itself ignores global word order. The vocabulary and the inverse document frequencies are learned from the training reviews only; the test reviews are merely transformed, which prevents information from the test set from leaking into the features.

Table 4: TF-IDF vectoriser configuration used in the notebook.

Parameter	Value	Effect
max_features	20,000	Retains the 20,000 most frequent terms (by corpus frequency); bounds feature dimensionality
ngram_range	(1, 2)	Unigrams and bigrams
min_df	2	Discards terms found in fewer than two documents (removes very rare strings and typographical noise)
max_df	0.95	Discards terms found in more than 95 % of documents
Other settings	scikit-learn defaults	Lower-casing, smoothed IDF, L2 normalisation, token pattern of two or more word characters
Output (train / test)	24,904 × 20,000 / 24,801 × 20,000	Sparse matrices; the first training review has 269 non-zero entries

To illustrate the representation, Table 5 lists the ten highest-weighted features of the first training review, as printed in the notebook. The terms “swedish”, “sex”, “yellow” and “curious” dominate because they are frequent in this review but comparatively rare in the corpus.

Table 5: Ten highest TF-IDF weights for the first training review.

Rank	Feature	TF-IDF weight
1	swedish	0.2157
2	sex	0.2112
3	yellow	0.2090
4	curious	0.1772
5	shown	0.1368
6	shown in	0.1253
7	sex and	0.1250
8	in	0.1180
9	issues	0.1100
10	am	0.1065

7.2 Logistic Regression

Logistic Regression is a linear probabilistic classifier for binary targets [8]. Given a feature vector x, it computes a linear score and converts it to a probability with the logistic (sigmoid) function:

$$z = w^{\top} x + b \tag{6}$$

$$P(y = 1 \vee x) = \sigma(z) = \frac{1}{1 + e^{-z}} \tag{7}$$

A review is predicted to be positive when P(y = 1 | x) ≥ 0.5, which is equivalent to z ≥ 0; the decision boundary is therefore a hyperplane in the 20,000-dimensional TF-IDF space. The weight vector w and intercept b are estimated by minimising the L2-regularised negative log-likelihood. Writing the labels as $\tilde{y}_i \in \{-1, +1\}$, the objective optimised by scikit-learn is

$$min_{w,b} \frac{1}{2} w^{\top} w + C \sum_{i=1}^{n} \log\left(1 + \exp\left(-\tilde{y}_i \left(w^{\top} x_i + b\right)\right)\right) \tag{8}$$

where C is the inverse regularisation strength. The notebook used C = 1.0, the default L2 penalty and the default lbfgs solver, with a maximum of 1,000 iterations. The model has 20,000 coefficients and one intercept, i.e. 20,001 learnable parameters. Each coefficient has a direct interpretation: a large positive weight on a term indicates evidence for the positive class, and a large negative weight indicates evidence for the negative class. The objective is convex, so the solver converges to the global optimum without sensitivity to initialisation.

7.3 Tokenisation and Padding

The deep-learning pipeline requires each review to be converted into a sequence of integers. The Keras Tokenizer was fitted on the training reviews with num_words = 20,000 and an out-of-vocabulary token ("<OOV>"). It assigns indices in order of decreasing word frequency, starting at 1 for the out-of-vocabulary token, and the 20,000-word limit means that only indices below 20,000 are retained; every other word is mapped to the out-of-vocabulary index. Index 0 is reserved for padding. On the full training set the tokenizer's word index contained 88,242 entries, which illustrates how much of the raw vocabulary is discarded by the 20,000-word cap.

Neural networks process batches of equal length, so each integer sequence was padded or truncated to MAX_LEN = 300 tokens using padding = "post" and truncating = "post". Shorter reviews are therefore extended with zeros at the end, and longer reviews retain only their first 300 tokens. The 300-token limit was chosen to cover the bulk of the length distribution (Section 5.3) while keeping the recurrent computation affordable. Post-padding also keeps the sequences right-padded, which is the layout required by the fast cuDNN LSTM kernels when masking is used.

7.4 Word Embeddings

Integer indices carry no notion of similarity. An embedding layer maps each index to a dense, real-valued vector, so that words used in similar contexts can acquire similar representations [12], [13]. If $E \in \mathbb{R}^{V \times d}$ is the embedding matrix, with vocabulary size V = 20,000 and embedding dimension d = 128, the word at position t with index x_t is represented by

$$e_t = E[x_t] \in \mathbb{R}^d \tag{9}$$

The embedding matrix contains 20,000 × 128 = 2,560,000 parameters. In this project it is initialised randomly and learned jointly with the rest of the network, rather than being initialised from pretrained vectors such as word2vec or GloVe. The layer was created with mask_zero = True, so that padded positions are flagged and ignored by the downstream recurrent layer.

7.5 LSTM and Bidirectional LSTM

A recurrent neural network processes a sequence one element at a time while carrying a hidden state. Plain recurrent networks suffer from vanishing and exploding gradients when the sequence is long. The Long Short-Term Memory network [9], [10] mitigates this by maintaining a separate cell state c_t that is modified through additive updates controlled by three gates. For an input vector x_t, previous hidden state h_{t-1} and previous cell state c_{t-1}, a standard LSTM cell computes

$$f_t = \sigma\left(W_f x_t + U_f h_{t-1} + b_f\right) \tag{10}$$

$$i_t = \sigma\left(W_i x_t + U_i h_{t-1} + b_i\right) \tag{11}$$

$$\tilde{c_t} = \tanh\left(W_c x_t + U_c h_{t-1} + b_c\right) \tag{12}$$

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c_t} \tag{13}$$

$$o_t = \sigma\left(W_o x_t + U_o h_{t-1} + b_o\right) \tag{14}$$

$$h_t = o_t \odot \tanh\left(c_t\right) \tag{15}$$

Here σ denotes the logistic sigmoid, tanh the hyperbolic tangent, and ⊙ element-wise multiplication. The forget gate f_t decides how much of the previous cell state is retained, the input gate i_t controls how much of the candidate state $\tilde{c}_t$ is written, and the output gate o_t determines how much of the cell state is exposed as the hidden state. Because the cell state is updated additively, gradients can propagate over many time steps, which allows the network to capture dependencies such as a negation early in a sentence that changes the meaning of a later adjective.

For a layer with u units and input dimension d, each direction has 4[u(d + u) + u] parameters (four weight blocks and four bias vectors):

$$P_{LSTM} = 4\left[u\left(d+u\right)+u\right] = 4\left[64\left(128+64\right)+64\right] = 49{,}408 \tag{16}$$

A Bidirectional LSTM [11] runs two independent LSTMs over the sequence: a forward network reading from the first to the last token and a backward network reading from the last to the first. With u = 64 units per direction, the representation returned by the layer is the concatenation of the final hidden state of each direction, a vector of 128 values:

$$h = \left[h_T^{\rightarrow}\,;\, h_1^{\leftarrow}\right] \tag{17}$$

The backward network therefore summarises the review starting from its end, while the forward network summarises it from its beginning, and every word contributes to the final representation in the context of both its left and right neighbours. The bidirectional layer has 2 × 49,408 = 98,816 parameters. Dropout [15] is then applied to this vector. During training each component is set to zero with probability p = 0.5 and the survivors are rescaled by 1/(1 − p); at inference time the layer is the identity:

$$\tilde{h} = \frac{\mu \odot h}{1-p}, \mu_j \;\; Bernoulli\left(1-p\right) \tag{18}$$

Finally, a single dense unit with a sigmoid activation produces the probability of the positive class:

$$\hat{y} = \sigma\left(w_o^{\top}\,\tilde{h} + b_o\right) \tag{19}$$

The dense layer has 128 + 1 = 129 parameters.

7.6 Training Objective and Optimisation

The network is trained by minimising the binary cross-entropy (log-loss) between the predicted probabilities and the true labels over a mini-batch of B reviews:

$$\mathscr{L} = -\frac{1}{B}\sum_{i=1}^{B}\left[y_i \log \hat{y}_i + \left(1-y_i\right)\log\left(1-\hat{y}_i\right)\right] \tag{20}$$

Optimisation uses Adam [14], which maintains exponentially decaying averages of the gradient g_t and of its square, with the standard decay rates $\beta_1 = 0.9$ and $\beta_2 = 0.999$, and applies bias-corrected, per-parameter step sizes:

$$m_t = \beta_1 m_{t-1} + \left(1-\beta_1\right) g_t, \; v_t = \beta_2 v_{t-1} + \left(1-\beta_2\right) g_t^2 \tag{21}$$

$$\theta_t = \theta_{t-1} - \alpha \frac{m_t / \left(1-\beta_1^t\right)}{\sqrt{v_t / \left(1-\beta_2^t\right)} + \varepsilon} \tag{22}$$

The Keras default learning rate α = 0.001 was used. Early stopping monitors the validation loss and halts training when it has not improved for a given number of epochs (the patience), restoring the weights of

the best epoch; this acts as a regulariser by preventing the network from continuing to fit noise in the training data.

7.7 Evaluation Metrics

Let TP, TN, FP and FN denote the numbers of true positives, true negatives, false positives and false negatives, with the positive class corresponding to label 1. The following metrics are used:

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN} \tag{23}$$

$$Precision = \frac{TP}{TP + FP}, Recall = \frac{TP}{TP + FN} \tag{24}$$

$$F_1 = \frac{2 \times Precision \times Recall}{Precision + Recall} \tag{25}$$

Precision measures how many of the reviews predicted as positive are in fact positive, recall measures how many of the actual positive reviews are found, and the F1-score is their harmonic mean. The confusion matrix tabulates all four counts and shows which type of error dominates.

8. Model Architecture

Figure 3 gives an overview of the complete experimental workflow. Both branches share the same data source, cleaning function and underlying 24,801 test reviews. However, the recorded BiLSTM test evaluation used a tokenizer whose word-index mapping differed from the mapping used during training; therefore, the two sets of predictions are not a valid like-for-like comparison.

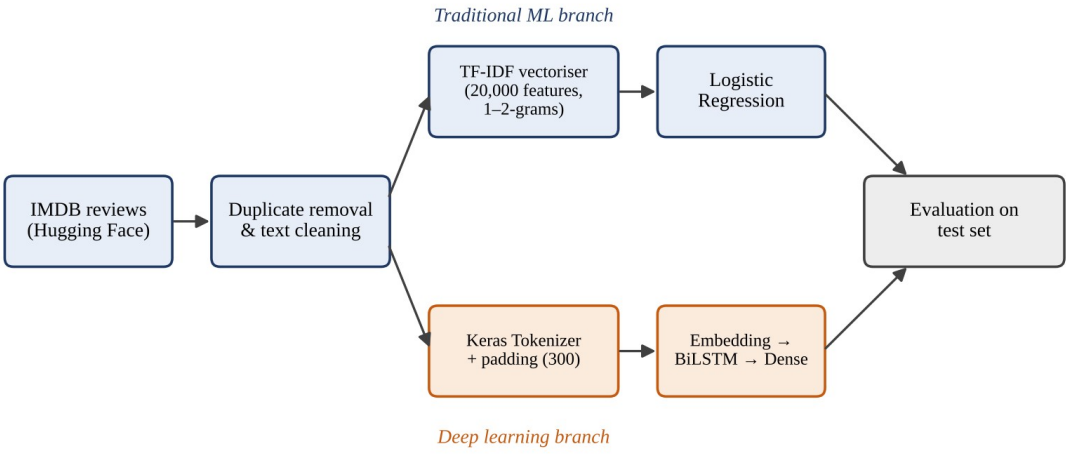


Figure 3: Overall experimental workflow of the two classification pipelines.

8.1 TF-IDF with Logistic Regression

The traditional pipeline consists of two stages: the TF-IDF vectoriser of Table 4, which converts each cleaned review into a 20,000-dimensional sparse vector, and a Logistic Regression classifier with 20,001 parameters that maps this vector to a probability of positive sentiment. The pipeline is fully linear in its features, trains on the complete training split of 24,904 reviews, and requires no validation partition because it has no iterative hyperparameter schedule that must be stopped.

8.2 Bidirectional LSTM

The deep-learning model is a Keras Sequential network with five layers, shown in Figure 4 and summarised in Table 6. The shapes and parameter counts are those printed by model.summary() in the notebook.

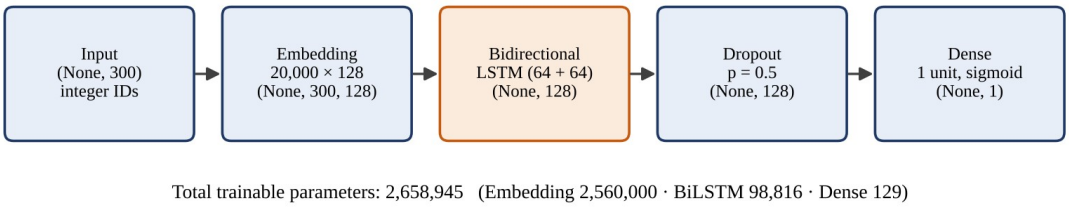


Figure 4: Architecture of the BiLSTM sentiment classifier.

Table 6: Layer-by-layer summary of the BiLSTM model (from model.summary()).

Layer	Configuration	Output shape	Parameters
Input	300 integer indices	(None, 300)	0
Embedding	20,000 × 128, mask_zero = True	(None, 300, 128)	2,560,000
Bidirectional(LSTM)	64 units per direction	(None, 128)	98,816
Dropout	rate 0.5	(None, 128)	0
Dense	1 unit, sigmoid	(None, 1)	129
Total (all trainable)			**2,658,945**

The model contains approximately 2.66 million trainable parameters (10.14 MB), more than 130 times the 20,001 parameters of the Logistic Regression model; 96 % of them reside in the embedding layer. It is compiled with the Adam optimiser, the binary cross-entropy loss and accuracy as the monitored metric.

9. Implementation Details

9.1 Software Environment

The experiments were implemented in Python in a Google Colab notebook with GPU acceleration enabled. Table 7 lists the main libraries and the versions that could be identified from the notebook outputs.

Table 7: Software environment (versions identified from notebook outputs).

Component	Role	Version / note
Python	Programming language	3.13
Hugging Face Datasets	Loading stanfordnlp/imdb	5.1.0 (pip log)
pandas, NumPy	Data handling	pandas 2.2.3, NumPy 2.4.6
Matplotlib, Seaborn	Visualisation	[version not recorded]
BeautifulSoup4	HTML removal	4.13.5

Component	Role	Version / note
scikit-learn	TF-IDF, Logistic Regression, metrics, data split	[version not recorded]
TensorFlow / Keras	Tokeniser, padding, BiLSTM	2.21.0
Accelerator	BiLSTM training and inference	GPU [model not recorded]
joblib	Persisting the TF-IDF model	—

9.2 Implementation Stages and Their Purpose

Table 8 describes each stage of the implementation and the reason it is needed.

Table 8: Implementation stages and their purpose.

Stage	Implementation	Purpose
Data loading	load_dataset("stanfordnlp/imdb"); conversion of the train and test splits into DataFrames	Obtain the labelled reviews in a convenient tabular form
Quality checks	Null count; duplicate detection on the text column; removal of duplicates in both splits	Remove redundant samples and verify integrity
EDA	Class counts, count plot, review-length statistics and histogram	Understand class balance and select the sequence length
Cleaning	preprocess_text() applied to every review	Remove noise while preserving sentiment-bearing words
TF-IDF features	TfidfVectorizer fitted on the training reviews, applied to both splits	Convert text to numeric vectors without test-set leakage
Logistic Regression	LogisticRegression(C = 1.0, max_iter = 1000) fitted on the training features	Learn a linear decision boundary
Tokenisation, padding	Keras Tokenizer (20,000 words, OOV token); pad_sequences to length 300	Convert text to fixed-length integer sequences
Train/validation split	train_test_split, test_size = 0.2, random_state = 42, stratified	Hold out data for early stopping and monitoring
Model definition	Sequential: Embedding → Bidirectional(LSTM(64)) → Dropout(0.5) → Dense(1, sigmoid)	Define the sequence classifier
Compilation, training	Adam, binary cross-entropy, batch size 64, up to 5 epochs, EarlyStopping on validation loss (patience 2, restore best weights)	Optimise the network and avoid over-fitting
Evaluation	Accuracy, precision, recall, F1, confusion matrix, classification report on the test set	Quantify and compare performance
Timing	time.time() around the TF-IDF + Logistic Regression fit and around model.fit()	Compare training cost
Custom inference	Four hand-written reviews cleaned and passed through both models	Qualitative check on unseen text
Persistence	joblib.dump for the vectoriser and classifier; model.save() for the BiLSTM (.keras)	Allow later re-use of the trained models

Chronology of the tokenisation stage. In the notebook the tokenizer was first fitted on the complete cleaned training split, and the resulting sequences were split 80/20 into training and validation partitions on which the BiLSTM was trained. A later cell re-fitted a new tokenizer on the 80 % training partition only, with the purpose of preventing validation-set vocabulary from influencing the encoding, and regenerated the training, validation and test sequences. The network was not trained again after this change. The consequences of this ordering for the reported BiLSTM test metrics are examined in Section 11.3.

10. Experimental Setup

Table 9 consolidates the configuration of the two models. The TF-IDF model was trained on all 24,904 cleaned training reviews. The BiLSTM was trained on 19,923 reviews, with 4,981 reviews used as the validation partition for monitoring and early stopping. Both pipelines were applied to the same underlying 24,801 test reviews, which were not used for model fitting or hyperparameter selection. The recorded BiLSTM test result is nevertheless invalid as an architectural comparison because the test sequences were generated with a different tokenizer mapping from the one used to train the network.

Table 9: Experimental configuration of the two models.

Parameter	TF-IDF + Logistic Regression	BiLSTM
Input representation	Sparse TF-IDF vector, unigrams + bigrams	Integer sequence, length 300
Vocabulary / features	20,000 (min_df = 2, max_df = 0.95)	20,000 words (+ OOV token)
Embedding dimension	Not applicable	128
Core model	Logistic Regression (L2, C = 1.0)	Bidirectional LSTM, 64 units per direction
Regularisation	L2 penalty	Dropout 0.5; early stopping
Optimiser	lbfgs (default), max_iter = 1000	Adam (default learning rate 0.001)
Loss function	Regularised log-loss	Binary cross-entropy
Batch size	Not applicable (full-batch solver)	64
Maximum epochs	Not applicable	5
Early stopping	Not applicable	Monitor val_loss, patience 2, restore best weights
Training reviews	24,904	19,923
Validation reviews	None	4,981 (stratified, random_state = 42)
Test reviews	24,801	24,801
Trainable parameters	20,001	2,658,945
Hardware	CPU (scikit-learn)	GPU (Google Colab)

The TF-IDF experiment was executed more than once in the notebook: once in an earlier session, which produced the classification report and confusion matrix, and again in a later session after the runtime had been restarted, which produced the values in the final comparison table. The two executions differ by a few test reviews (test accuracy 0.8963 and 0.8962 respectively). The final-comparison values are used in the summary tables of this report, and the confusion matrix and per-class report originate from the earlier execution. Random seeds for TensorFlow were not set in the notebook, so the BiLSTM results are not guaranteed to be bit-for-bit reproducible.

11. Results and Performance Evaluation

11.1 TF-IDF with Logistic Regression

On the 24,801 test reviews the TF-IDF + Logistic Regression model reached an accuracy of 0.8962, a precision of 0.8920, a recall of 0.9023 and an F1-score of 0.8971 for the positive class. The per-class classification report is given in Table 10. Performance is nearly symmetric across the two classes, with F1-scores of about 0.90 for both.

Table 10: Classification report of TF-IDF + Logistic Regression on the test set (as printed in the notebook).

Class	Precision	Recall	F1-score	Support
Negative	0.90	0.89	0.90	12,361
Positive	0.89	0.90	0.90	12,440
Accuracy			0.90	24,801
Macro average	0.90	0.90	0.90	24,801
Weighted average	0.90	0.90	0.90	24,801

11.2 BiLSTM Training Behaviour

The BiLSTM was trained for up to five epochs with early stopping on the validation loss. Training stopped after the fourth epoch because the validation loss had not improved on the epoch-2 value for two consecutive epochs, and the weights of the best epoch (epoch 2) were restored. Table 11 reproduces the training log and Figure 5 plots the curves; the curves were redrawn from the logged values, since the notebook only plotted the accuracy curves.

Table 11: BiLSTM training log (values from the notebook output).

Epoch	Train accuracy	Train loss	Val. accuracy	Val. loss	Time (s)
1	0.7734	0.4679	0.8151	0.4034	31
2	**0.9087**	**0.2337**	**0.8759**	**0.2999**	**14**
3	0.9390	0.1647	0.8522	0.3857	10
4	0.9692	0.0920	0.8661	0.4015	9

Epoch 2 (bold) had the lowest validation loss and its weights were restored. Total fitting time: 64.45 s.

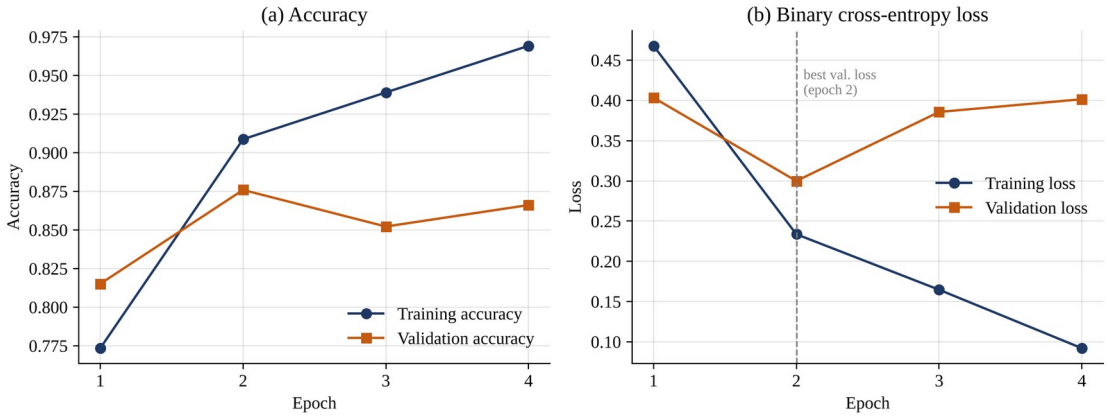


Figure 5: BiLSTM training and validation accuracy (a) and loss (b), redrawn from the logged values.

The training accuracy increases monotonically from 77.3 % to 96.9 % and the training loss falls from 0.468 to 0.092, whereas the validation accuracy peaks at 87.6 % in epoch 2 and then fluctuates (85.2 % and 86.6 %), and the validation loss rises from 0.300 to 0.402 after epoch 2. This widening gap between training and validation behaviour is the typical signature of over-fitting: the network, with more than 2.6 million parameters, begins to memorise the training reviews from the third epoch onward. Early stopping with weight restoration therefore selected an appropriate point, and the dropout layer alone was not sufficient to prevent the divergence. The first epoch is also the slowest (31 s) because it includes the one-time set-up of the GPU kernels; later epochs take 9 to 14 s.

11.3 BiLSTM Test Evaluation and Validity of the Recorded Result

The restored BiLSTM was evaluated on the test set. The metrics printed by the notebook are an accuracy of 0.5714, a precision of 0.5779, a recall of 0.5400 and an F1-score of 0.5583 (Table 12). Predicting 24,801 test reviews took 16 s.

Table 12: Classification report of the BiLSTM on the test set, as recorded in the notebook.

Class	Precision	Recall	F1-score	Support
Negative	0.57	0.60	0.58	12,361
Positive	0.58	0.54	0.56	12,440
Accuracy			0.57	24,801
Macro average	0.57	0.57	0.57	24,801
Weighted average	0.57	0.57	0.57	24,801

Important note on the validity of these BiLSTM test metrics

These figures should be treated as provisional. As described in Section 9.2, the network was trained on sequences produced by a tokenizer fitted on the complete training split, but the test sequences were afterwards produced by a different tokenizer that was re-fitted on the 80 % training partition only. The Keras Tokenizer assigns word indices by descending frequency within the data it is fitted on, so two tokenizers fitted on different data do not in general give the same word-to-index mapping. Consequently the embedding rows learned during training are not guaranteed to correspond to the same words at test time.

The size of the discrepancy supports this interpretation: the restored network reached 87.59 % accuracy on its validation partition (encoded with the original tokenizer) but only 57.14 % on the test set (encoded with the re-fitted tokenizer). The recorded test result therefore measures the combination of the trained network and an inconsistent input encoding, and it should not be interpreted as the intrinsic ability of the BiLSTM architecture. The reported custom-review predictions of the BiLSTM (Section 13) were produced with the same re-fitted tokenizer and are affected in the same way.

Retraining the BiLSTM after the tokenizer has been fitted on the training partition, and then evaluating it on the test set with that same tokenizer, would give a valid test result. Table 15 and the following sections show the recorded values together with this caveat. [To be replaced with re-run results if available.]

Table 13: Additional BiLSTM validation information from the training log.

Quantity (validation partition, epoch 2 weights)	Value	Source
Accuracy	0.8759	Training log
Loss (binary cross-entropy)	0.2999	Training log
Precision, recall, F1-score	[Not recorded]	—
Number of validation reviews	4,981	Derived from the 80/20 split

The validation accuracy is reported only as supporting information. It was computed on a smaller partition and used for early stopping, so it is not a substitute for an independent test result. In addition, the tokenizer had initially been fitted before the train/validation split, meaning that its vocabulary statistics had seen the validation texts (but not their labels). A clean rerun should fit the tokenizer only on the training partition before encoding training, validation and test reviews.

11.4 Confusion Matrices

Figure 6 shows the confusion matrices produced in the notebook and Table 14 lists the counts and the derived error rates. For the TF-IDF model, 11,005 negative and 11,225 positive reviews were classified correctly, with 1,356 negative reviews misclassified as positive (false positives) and 1,215 positive reviews misclassified as negative (false negatives). The errors are almost symmetric: the false-positive rate is 11.0 % and the false-negative rate is 9.8 %. The recorded BiLSTM matrix shows 7,454 true negatives, 6,718 true positives, 4,907 false positives and 5,722 false negatives, corresponding to a false-positive rate of 39.7 % and a false-negative rate of 46.0 %; its errors are large for both classes, which is consistent with an input encoding that is no longer aligned with the learned embeddings (Section 11.3) rather than with a systematic bias towards one class.

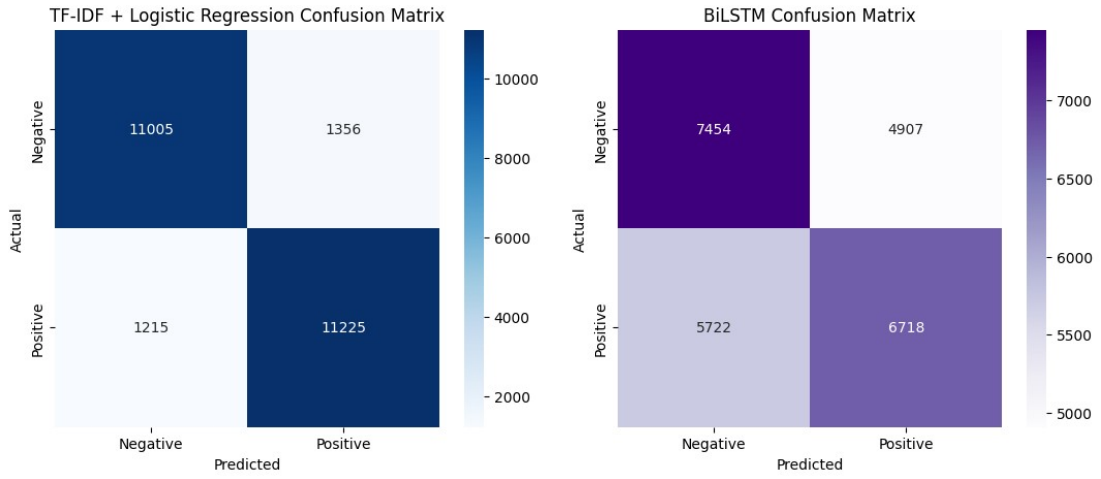


Figure 6: Confusion matrices on the test set: TF-IDF + Logistic Regression (left) and BiLSTM, recorded evaluation (right; see Section 11.3).

Table 14: Confusion-matrix counts and derived error rates on the test set (positive class = 1).

Model	TN	FP	FN	TP	False-positive rate	False-negative rate
TF-IDF + Logistic Regression	11,005	1,356	1,215	11,225	11.0 %	9.8 %
BiLSTM (recorded)	7,454	4,907	5,722	6,718	39.7 %	46.0 %

Rates were computed from the counts: FPR = FP / (FP + TN), FNR = FN / (FN + TP). The TF-IDF matrix is from the earlier execution (accuracy 0.8963), see Section 10.

11.5 Overall Performance Comparison

Table 15 and Figure 7 display the recorded metrics side by side for transparency. They must not be interpreted as a fair performance comparison: the BiLSTM test metrics are diagnostic only because the test tokenizer mapping did not match the training mapping.

Table 15: Performance of the two models on the test set (recorded values).

Evaluation metric	TF-IDF + Logistic Regression	BiLSTM (invalid tokenizer mapping; diagnostic only)	Difference
Accuracy	0.8962	0.5714	Not comparable
Precision (positive class)	0.8920	0.5779	Not comparable
Recall (positive class)	0.9023	0.5400	Not comparable
F1-score (positive class)	0.8971	0.5583	Not comparable
Training time (s)	25.34	64.45	Not comparable

The BiLSTM metrics in this table are diagnostic only and are not comparable with the TF-IDF test metrics because of the tokenizer mismatch described in Section 11.3. Training times were also measured on different hardware and cover different stages (Section 11.6).

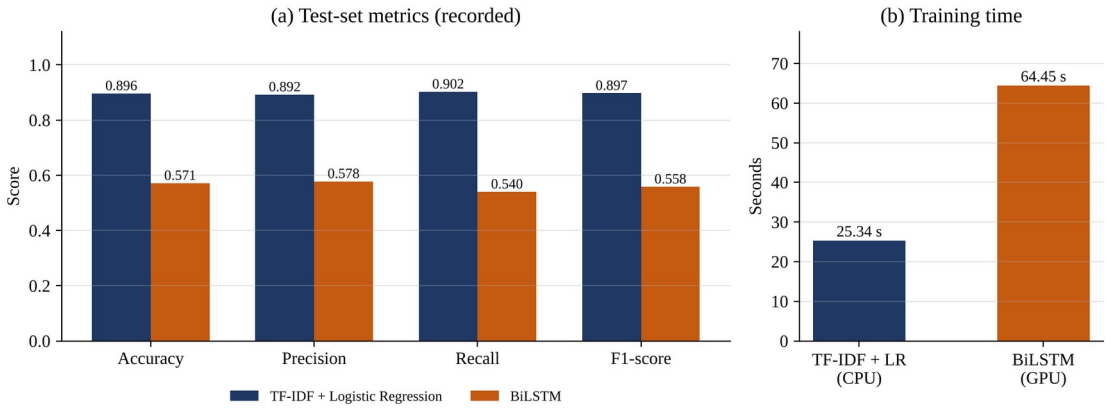


Figure 7: Comparison of test-set metrics (a) and training time (b); BiLSTM values as recorded in the notebook.

The recorded TF-IDF metrics are valid for the TF-IDF pipeline. The recorded BiLSTM test metrics are not valid for judging the trained network because the test tokenizer mapping differed from the training mapping. Therefore, the numerical gap shown in Table 15 is not an interpretable model-performance difference and no ranking of the two architectures can be concluded from it.

11.6 Computational Efficiency and Training Time

Table 16 summarises the computational characteristics. The TF-IDF stage and the classifier were timed together (vectorisation of both splits plus fitting), whereas the BiLSTM timing covers only model.fit() and excludes tokenisation and padding.

Table 16: Computational characteristics of the two models.

Characteristic	TF-IDF + Logistic Regression	BiLSTM
Training time (recorded)	25.34 s (vectorisation + fit)	64.45 s (4 epochs of model.fit)
Hardware used	CPU	GPU
Time per epoch	Not applicable	31 s, 14 s, 10 s, 9 s
Prediction time on 24,801 test reviews	[Not recorded]	16 s
Trainable parameters	20,001	2,658,945 (about 133 times more)
Training samples	24,904	19,923
Hyperparameter sensitivity	Low (few settings, convex problem)	High (architecture, initialisation, epochs)
Wall-clock ratio	1.00	2.54 × TF-IDF

The BiLSTM required about 2.5 times the wall-clock time of the TF-IDF pipeline even though it ran on a GPU and a smaller number of training samples, while the TF-IDF pipeline ran on a CPU. The comparison is therefore not hardware-controlled, and since the TF-IDF figure also includes feature extraction, it is indicative rather than exact. The practical conclusion is nevertheless clear: the linear model is cheaper to train, has a far smaller memory footprint and has fewer settings to tune.

12. Comparative Analysis: TF-IDF versus BiLSTM

Table 17 contrasts the two approaches along the dimensions identified in the objectives. Quantitative entries are taken from Section 11; the remaining entries describe general properties of the methods.

Table 17: Comparison of TF-IDF + Logistic Regression and BiLSTM.

Aspect	TF-IDF + Logistic Regression	BiLSTM
Text representation	Sparse, high-dimensional, statistical; word and bigram frequencies weighted by rarity	Dense learned embeddings; sequence of 300 token vectors
Word order and context	Only local order through bigrams; no long-range context	Models order and context in both directions through gated recurrence
Semantics	Words are independent features; no notion of similarity	Embeddings can place related words close together if learned from enough data
Recorded test accuracy	0.8962	0.5714 (provisional, Section 11.3)
Validation accuracy	Not applicable	0.8759 at the best epoch
Over-fitting behaviour	Controlled by L2 penalty; convex objective	Over-fits after epoch 2; requires dropout and early stopping
Training cost	25.34 s (CPU)	64.45 s (GPU)
Model size	20,001 parameters	2,658,945 parameters
Interpretability	High: coefficients per term	Low: distributed representations

Aspect	TF-IDF + Logistic Regression	BiLSTM
Pipeline fragility	Few moving parts	Index mapping, padding and truncation must remain consistent between training and inference

Representation. TF-IDF converts a review into a weighted bag of words and bigrams. The representation is fixed once the vocabulary is chosen, which makes the learning problem convex and cheap. For polarised film reviews, where sentiment is largely carried by individual evaluative words ("excellent", "boring", "waste") and short phrases, this representation captures much of the available signal; this agrees with the established observation that linear models over unigram and bigram features are strong baselines on this dataset [3], [16].

Sequential modelling. The BiLSTM can in principle exploit information that the bag-of-words model cannot, such as negation scope, contrast between clauses and the position of a verdict in a review. However, this capacity has to be learned from the data, together with the 2.56 million embedding parameters, from 19,923 training reviews and without pretrained word vectors. The training log shows that the model fits the training data very closely (96.9 % accuracy in epoch 4) but generalises only to about 87 % on validation data, so extra capacity was not converted into a clear generalisation advantage in this configuration.

Reliability of conclusions. A fair comparison requires that both models are evaluated with identical, consistent preprocessing. This holds for the TF-IDF pipeline, in which the vectoriser used for training is also used for testing. It does not hold for the recorded BiLSTM test evaluation (Section 11.3). A definitive ranking of the two approaches therefore requires a repeated BiLSTM evaluation; this report limits itself to the observation that, on the results available, TF-IDF + Logistic Regression is the stronger and cheaper model.

13. Custom Review Predictions

Both trained models were applied to four reviews written by the authors that are not part of the IMDB dataset. Each review was passed through the same cleaning function and encoded with the respective pipeline (TF-IDF vectoriser; Keras tokenizer and padding to 300 tokens). Table 18 lists the outputs printed by the notebook. The expected polarity column is the authors' own reading of each sentence and not a dataset label.

Table 18: Predictions of the two models on custom reviews.

#	Review text	Expected polarity	TF-IDF + LR	BiLSTM
1	"This movie was absolutely fantastic. I loved it."	Positive	Positive	Negative
2	"The movie was boring and a complete waste of time."	Negative	Negative	Negative
3	"The acting was good but the story was terrible."	Mixed	Negative	Positive
4	"One of the best movies I have ever watched."	Positive	Positive	Positive

The TF-IDF model classified all three unambiguous reviews as expected. For the mixed review (No. 3), in which a positive clause is contrasted with a strongly negative one, it predicted Negative, which is a

reasonable outcome because a strongly negative term such as “terrible” typically carries a large coefficient; no ground truth exists for this sentence. The BiLSTM classified Review 1 as Negative, which contradicts the clearly positive wording, and classified Review 3 as Positive; it agreed with the expected polarity on Reviews 2 and 4. These BiLSTM predictions were generated with the re-fitted tokenizer described in Section 11.3 and therefore cannot be used to judge the architecture. In any case, four sentences form an anecdotal sample and no statistical conclusion should be drawn from them. Output probabilities were not printed in the notebook; recording them would be informative for the mixed review in particular.

14. Discussion

A strong linear baseline. TF-IDF with Logistic Regression reached about 89.6 % test accuracy with balanced precision and recall and with errors spread almost evenly between the classes. It trained in about 25 s on a CPU. These results are in line with the literature, in which bigram-augmented linear models are competitive on IMDB [16]. For practitioners, this model offers high accuracy, low cost and the ability to inspect the most influential terms.

Learning dynamics of the BiLSTM. The training log demonstrates that the BiLSTM learns the task: validation accuracy reached 87.6 % after only two epochs. At the same time, the rising validation loss after epoch 2 and the near-perfect training accuracy illustrate the over-fitting tendency of a large network trained from randomly initialised embeddings on 20,000 reviews. Measures that could improve generalisation include pretrained embeddings, stronger regularisation, a smaller embedding dimension and a lower learning rate.

Pipeline consistency matters. The most instructive finding of this study is methodological. A drop from 87.6 % validation accuracy to 57.1 % test accuracy is far larger than any plausible generalisation gap between two samples of the same distribution, and the notebook's order of operations offers a direct explanation: the tokenizer used at test time differs from the tokenizer used at training time. This illustrates that, in deep-learning text pipelines, the preprocessing artefacts (vocabulary, index mapping, padding convention) form part of the model and must be fitted once, stored, and reused unchanged for validation, testing and deployment. The TF-IDF pipeline is less exposed to this risk because the fitted vectoriser object is applied directly to new text.

Truncation. Post-truncation retains the first 300 tokens of a review. Reviews often conclude with an overall verdict, so truncating the end may discard informative text for the longest reviews; pre-truncation, or a model that reads both the beginning and the end, could be considered. The effect was not measured in this project.

Suggested remedial work. (i) Fit the tokenizer on the training partition, regenerate all sequences, retrain the BiLSTM and evaluate it on the test set using the same tokenizer; (ii) save the tokenizer together with the model; (iii) repeat the training with several random seeds and report the mean and spread; (iv) record the prediction time of both models on the same hardware; and (v) optionally evaluate pretrained embeddings and pretrained transformer encoders.

15. Limitations

1. **Evaluation inconsistency.** The recorded BiLSTM test metrics were obtained with a tokenizer different from the one used during training (Section 11.3) and are therefore not a valid measure of the architecture.
2. **Modified test split.** Removal of 199 duplicate reviews from the test split means that results are not directly comparable with published results on the standard 25,000-review test set.

3. **Single run.** Each model was trained and evaluated once. No random seeds were fixed for TensorFlow, no repeated runs or confidence intervals were produced, and the small difference between two TF-IDF executions (0.8963 and 0.8962) suggests that small run-to-run or environment-related variation exists.
4. **Limited hyperparameter exploration.** The configurations (20,000 features, C = 1.0, 128-dimensional embeddings, 64 units, dropout 0.5, sequence length 300) were fixed in advance and were not tuned.
5. **Truncation.** Reviews longer than 300 tokens lose information; the TF-IDF model has no such restriction, so the two models do not see exactly the same content for long reviews.
6. **Random embeddings.** The BiLSTM used randomly initialised, trainable embeddings rather than pretrained word vectors, which is a disadvantage when the training set is of moderate size.
7. **Unequal timing conditions.** The two training times were measured on different hardware (CPU and GPU) and cover different stages, so they are indicative only. Prediction time was recorded for the BiLSTM only.
8. **Binary, domain-specific task.** The dataset contains only strongly polarised movie reviews; the conclusions may not transfer to neutral, sarcastic or multi-aspect text, or to other domains.
9. **Small custom-review sample.** Four hand-written sentences provide only anecdotal evidence.

16. Conclusion

This project implemented and compared two approaches for binary sentiment classification of IMDB movie reviews: TF-IDF features with Logistic Regression, and a Bidirectional LSTM with trainable word embeddings. After duplicate removal, the underlying test collection contained 24,801 reviews. The TF-IDF model was evaluated consistently on this cleaned set; the recorded BiLSTM test metrics are provisional diagnostic outputs because of the tokenizer mismatch. In the final recorded TF-IDF execution, the pipeline achieved an accuracy of 0.8962, a precision of 0.8920, a recall of 0.9023 and an F1-score of 0.8971, with a training time of 25.34 s. A separate earlier TF-IDF execution produced the classification report and confusion-matrix counts shown in Tables 10 and 14; those earlier-run counts should not be used to recompute or validate the final-run metrics.

The BiLSTM, with 2,658,945 parameters, learned the task during training and reached a best validation accuracy of 0.8759 before beginning to over-fit; early stopping restored the best epoch. Its recorded test metrics (accuracy 0.5714, F1-score 0.5583) were, however, produced with a tokenizer that differed from the one used for training, so they do not provide a valid estimate of the architecture's test performance. Accordingly, no claim is made here that either architecture is intrinsically superior; the only supported conclusions are that TF-IDF + Logistic Regression is a strong, inexpensive and robust baseline on this dataset, and that the BiLSTM requires a carefully consistent preprocessing pipeline and further tuning before it can be compared fairly.

A valid head-to-head comparison remains incomplete. The BiLSTM must be retrained from scratch after splitting the cleaned training reviews into training and validation partitions, fitting one tokenizer on the training partition only, and using that exact tokenizer for all training, validation, test and custom-review sequences. The original 25,000-review test split should also be retained for a benchmark-aligned evaluation, with any duplicate-removal experiment reported separately. After this correction, the two models can be compared fairly. Repeated runs with fixed random seeds and pretrained embeddings are useful further experiments.

References

[1] A. L. Maas, R. E. Daly, P. T. Pham, D. Huang, A. Y. Ng and C. Potts, “Learning word vectors for sentiment analysis,” in Proc. 49th Annual Meeting of the Association for Computational Linguistics: Human Language Technologies, 2011, pp. 142–150.

[2] B. Pang and L. Lee, “Opinion mining and sentiment analysis,” Foundations and Trends in Information Retrieval, vol. 2, no. 1–2, pp. 1–135, 2008.

[3] B. Pang, L. Lee and S. Vaithyanathan, “Thumbs up? Sentiment classification using machine learning techniques,” in Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP), 2002, pp. 79–86.

[4] B. Liu, Sentiment Analysis and Opinion Mining. San Rafael, CA: Morgan & Claypool, 2012.

[5] K. Spärck Jones, “A statistical interpretation of term specificity and its application in retrieval,” Journal of Documentation, vol. 28, no. 1, pp. 11–21, 1972.

[6] G. Salton and C. Buckley, “Term-weighting approaches in automatic text retrieval,” Information Processing & Management, vol. 24, no. 5, pp. 513–523, 1988.

[7] C. D. Manning, P. Raghavan and H. Schütze, Introduction to Information Retrieval. Cambridge, U.K.: Cambridge University Press, 2008.

[8] C. M. Bishop, Pattern Recognition and Machine Learning. New York, NY: Springer, 2006.

[9] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” Neural Computation, vol. 9, no. 8, pp. 1735–1780, 1997.

[10] F. A. Gers, J. Schmidhuber and F. Cummins, “Learning to forget: Continual prediction with LSTM,” Neural Computation, vol. 12, no. 10, pp. 2451–2471, 2000.

[11] M. Schuster and K. K. Paliwal, “Bidirectional recurrent neural networks,” IEEE Transactions on Signal Processing, vol. 45, no. 11, pp. 2673–2681, 1997.

[12] T. Mikolov, K. Chen, G. Corrado and J. Dean, “Efficient estimation of word representations in vector space,” arXiv:1301.3781, 2013.

[13] J. Pennington, R. Socher and C. D. Manning, “GloVe: Global vectors for word representation,” in Proc. EMNLP, 2014, pp. 1532–1543.

[14] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in Proc. International Conference on Learning Representations (ICLR), 2015; arXiv:1412.6980.

[15] N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever and R. Salakhutdinov, “Dropout: A simple way to prevent neural networks from overfitting,” Journal of Machine Learning Research, vol. 15, pp. 1929–1958, 2014.

[16] S. Wang and C. D. Manning, “Baselines and bigrams: Simple, good sentiment and topic classification,” in Proc. 50th Annual Meeting of the Association for Computational Linguistics (Short Papers), 2012, pp. 90–94.

[17] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” Journal of Machine Learning Research, vol. 12, pp. 2825–2830, 2011.

[18] M. Abadi et al., “TensorFlow: A system for large-scale machine learning,” in Proc. 12th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 2016, pp. 265–283.

[19] F. Chollet et al., “Keras,” 2015. [Online]. Available: https://keras.io

[20] I. Goodfellow, Y. Bengio and A. Courville, Deep Learning. Cambridge, MA: MIT Press, 2016.

[21] Q. Lhoest et al., “Datasets: A community library for natural language processing,” in Proc. EMNLP 2021: System Demonstrations, 2021, pp. 175–184.

[22] Hugging Face, “stanfordnlp/imdb” dataset card. [Online]. Available: https://huggingface.co/datasets/stanfordnlp/imdb